

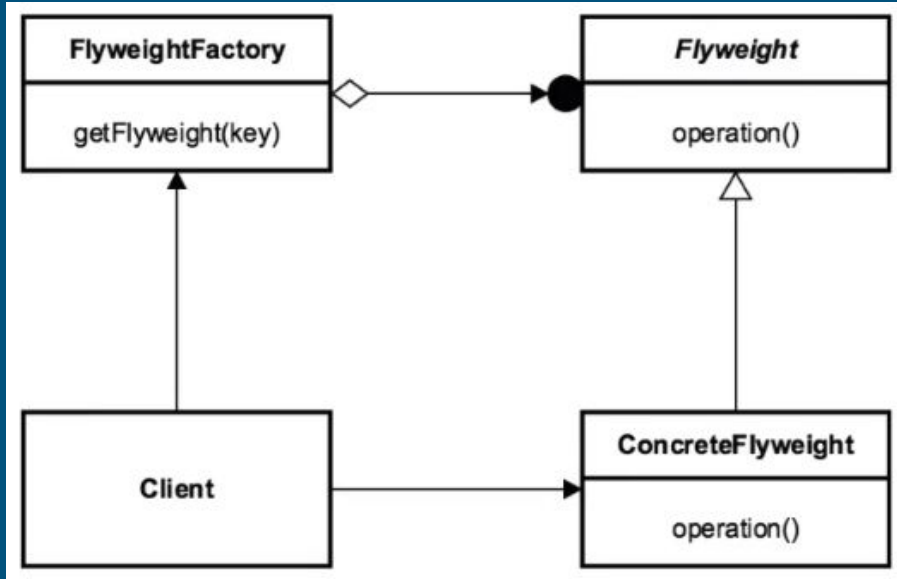


Design Pattern Flyweight



Ana Jesus
11/04/2023





What is Flyweight Design Pattern

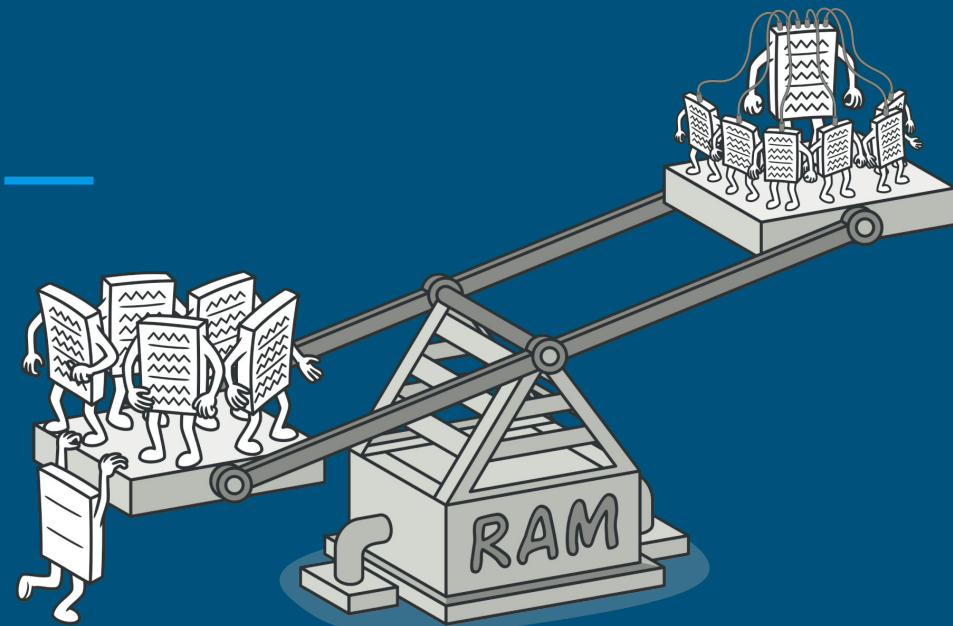
Why?



- Design to control the object creation
 - Allow to create one object per type
 - If you want a object with the same property(already created) return the same object and nor create a new one
-

What's the Problem

- Too many objects can slow things down
- Too many objects might consume a large piece of memory and can slow down the application



X-Programming

- ❖ Site that allows users to create and execute programs using programming language
- ❖ Choose one language, write the program and execute to see the result

```
//set the code done by the programmer in order to get it executed
// property code with setter and getter
3 usages
public class Code {
    2 usages
    String code;

    no usages
    public String getCode() { return code; }

    no usages
    public void setCode(String code) { this.code = code; }
}
```

X-Programming Interface

```
//one method execute witch takes the Code Object as parameter
```

```
2 usages 1 implementation
```

```
public interface Platform {
```

```
    1 usage 1 implementation
```

```
    public void execute(Code code);
```

```
}
```

```
,
```

X-Programming - JavaPlatform

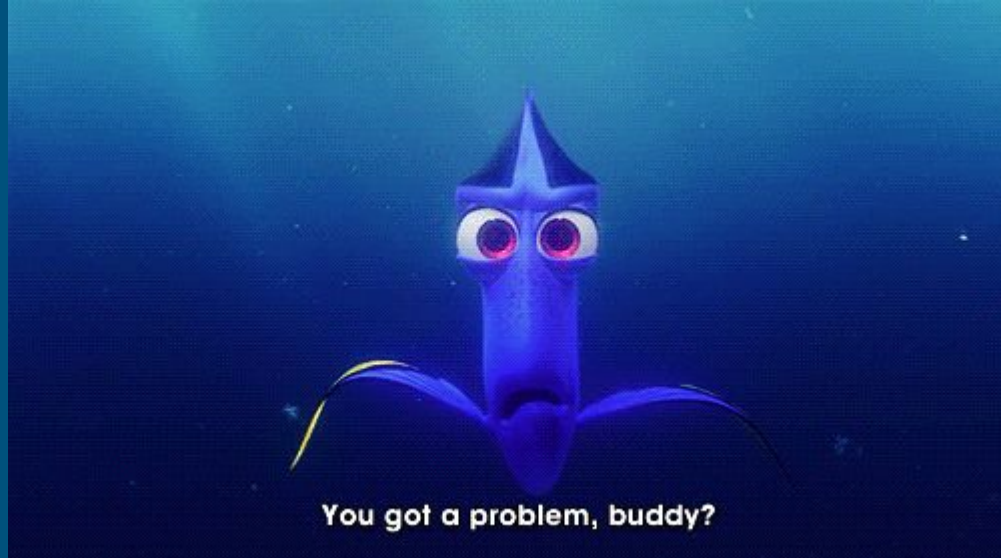
```
//implement interface Platform and provides an implementation for
// the method to execute code in Java
1 usage
public class JavaPlatform implements Platform{

    1 usage
    public JavaPlatform(){
        System.out.println("JavaPlatform Object created");
    }

    1 usage
    @Override
    public void execute(Code code) {
        System.out.println("Compiling and Executing Java code");
    }
}
```

```
/Users/ana/Library/Java/JavaVirtualMach  
JavaPlatform Object created  
Compiling and Executing Java code  
  
Process finished with exit code 0
```

The Result



Flyweight Factory

```
no usages
public final class PlatformFactory {
    2 usages
    private static Map<String, Platform> map = new HashMap<>();
    no usages
    private PlatformFactory(){
        throw new AssertionError( detailMessage: "Cannot instantiate the class");
    }
    no usages 4 related problems
    public static synchronized Platform getInstance(String platformType){
        Platform platform = map.get(platformType);
        if(platform==null){
            switch(platformType){
                case "C" : platform = new CPlatform();
                    break;
                case "JavaScript" : platform = new JavaScriptPlatform();
                    break;
                case "JAVA" : platform = new JavaPlatform();
                    break;
            }
            map.put(platformType, platform);
        }
        return platform;
    }
}
```

Flyweight Main

```
public class Main {  
    no usages  
    public static void main(String[] args) {  
  
        Code code = new Code();  
        code.setCode("C Code...");  
        Platform platform = new PlatformFactory.getPlatformFactory("C");  
        platform.execute(code);  
        System.out.println("*****");  
  
        code = new Code();  
        code.setCode("JAVA Code...");  
        platform = PlatformFactory.getPlatformInstance("JAVA");  
        platform.execute(code);  
        System.out.println("*****");  
  
        code = new Code();  
        code.setCode("JavaScript Code...");  
        platform = PlatformFactory.getPlatformInstance("JavaScript");  
        platform.execute(code);  
    }  
}
```



The outcome



```
CPlatform object created
Compiling and executing C code.
*****
Compiling and executing C code.
*****
JavaPlatform object created
Compiling and executing Java code.
*****
Compiling and executing Java code.
```

Expected outcome



Thank you and Goodbye